

Two orthogonal view of a software

There are two orthogonal views of software development.

1. Traditional Technique – focuses on **data and functions**.
2. Object Oriented methodologies – focuses on objects that **combines data and functionality**.

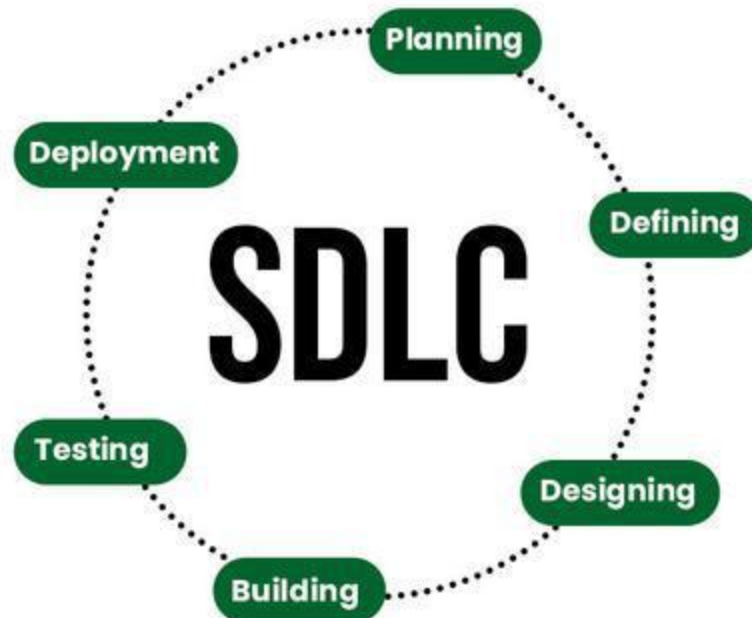
Object oriented systems development develop software by building objects that **can be easily replaced, modified and reused**

Software Development Life Cycle (SDLC)

- **(SDLC) is a structured process that is used to design, develop, and test good-quality software.**
- SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step.
- The **goal of the SDLC life cycle model** is to deliver high-quality, maintainable software that meets the user's requirements.
- SDLC in software engineering models outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets users requirements

Software Development Life Cycle (SDLC)

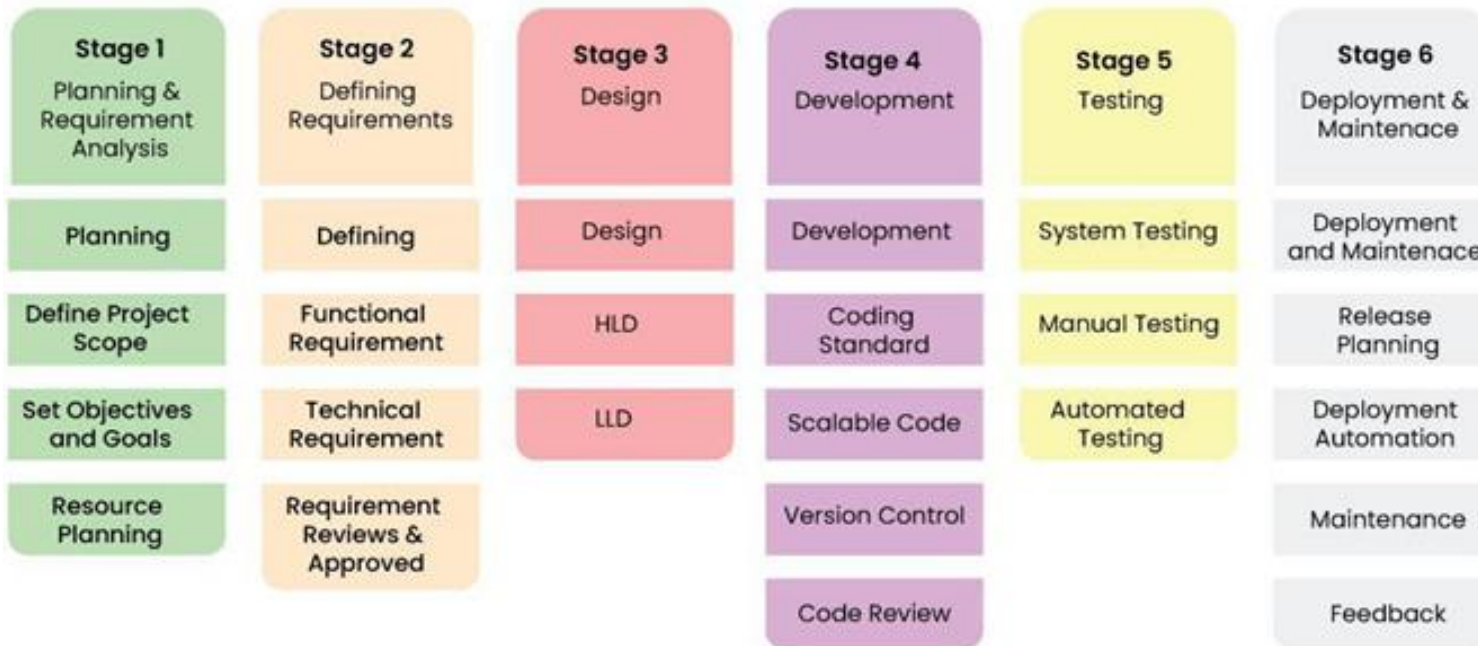
- **SDLC is a process followed for software building within a software organization.** SDLC consists of a precise plan that describes how to develop, maintain, replace, and enhance specific software.
- The life cycle defines a method for improving the quality of software and the all-around development process.



Stages of the Software Development Life Cycle

- SDLC specifies the task(s) to be performed at various stages by a software engineer or developer.
- It ensures that the end product is able to meet the customer's expectations and fits within the overall budget. Hence, it's vital for a software developer to have prior knowledge of this software development process.
- SDLC is a collection of these six stages, and the stages of SDLC are as follows:

Stages of the Software Development Life Cycle



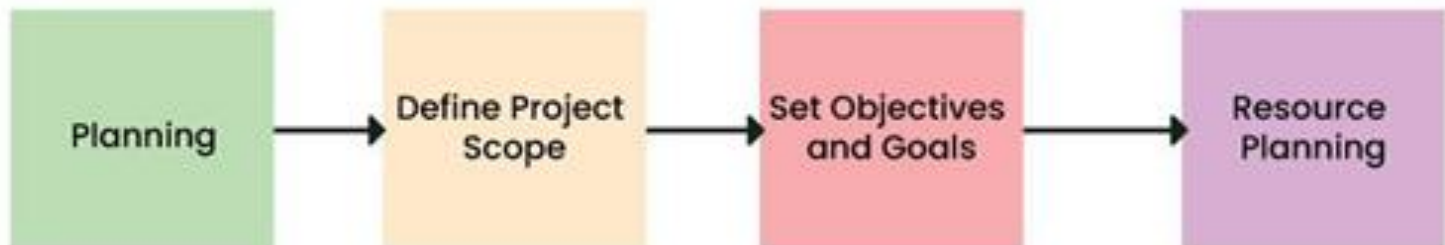
Stages of the Software Development Life Cycle

The [SDLC Model](#) involves six phases or stages while developing any software.

- **Stage-1: Planning and Requirement Analysis**
- Planning is a crucial step in everything, just as in [software development](#).
- In this same stage, [requirement analysis](#) is also performed by the developers of the organization. This is attained from customer inputs, and sales department/market surveys.
- The information from this analysis forms the building blocks of a basic project.
 - The quality of the project is a result of planning.
- Thus, in this stage, the basic project is designed with all the available information.

Cont..

Stage-1: Planning and Requirement Analysis



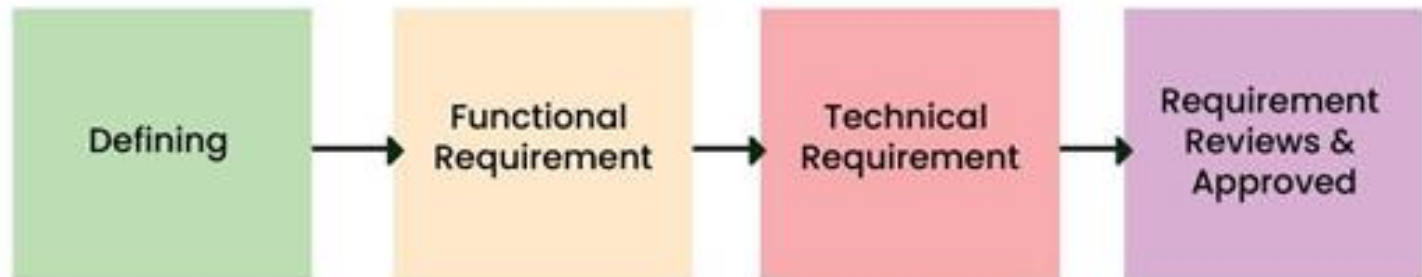
Cont..

Stage-2: Defining Requirements

- In this stage, all the requirements for the target software are specified. These requirements get approval from customers, market analysts, and stakeholders.
- This is fulfilled by utilizing SRS (Software Requirement Specification). This is a sort of document that specifies all those things that need to be defined and created during the entire project cycle.

Cont..

Stage-2: Defining Requirements



Cont..

- **Stage-3: Designing Architecture**
- [SRS](#) is a reference for software designers to come up with the best architecture for the software. Hence, with the requirements defined in SRS, multiple designs for the product architecture are present in the Design Document Specification (DDS).
- This DDS is assessed by market analysts and stakeholders. After evaluating all the possible factors,
- the most practical and logical design is chosen for development.

Cont..

Stage-3: Designing Architecture

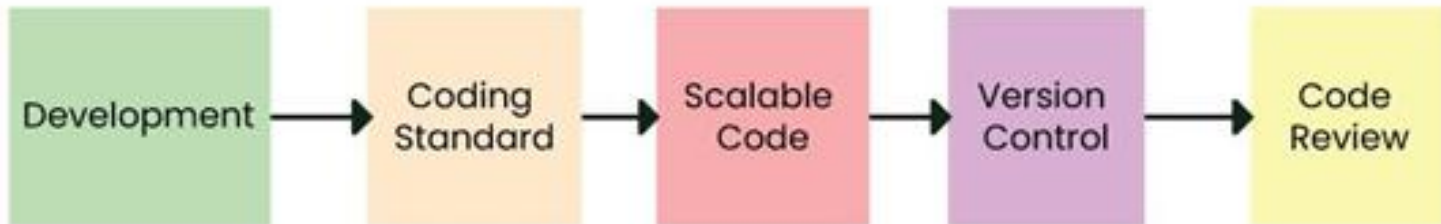


Cont..

- **Stage-4: Developing Product**
- At this stage, the fundamental development of the product starts. For this, developers use a specific programming code as per the design in the DDS.
 - Hence, it is important for the coders to follow the protocols set by the association. Conventional programming tools like compilers, interpreters, debuggers, etc. are also put into use at this stage.
 - Some popular languages like C/C++, Python, Java, etc. are put into use as per the software regulations.

Cont..

Stage-4: Developing Product



Cont..

Stage-5: Product Testing and Integration

- After the development of the product, testing of the software is necessary to ensure its smooth execution. Although, minimal testing is conducted at every stage of SDLC.
 - Therefore, at this stage, all the probable flaws are tracked, fixed, and retested. This ensures that the product confronts the quality requirements of SRS.

Cont..

- **Documentation, Training, and Support:** [Software documentation](#) is an essential part of the software development life cycle. A well-written document acts as a tool and means to information repository necessary to know about software processes, functions, and maintenance.
- Documentation also provides information about how to use the product.
 - Training in an attempt to improve the current or future employee performance by increasing an employee's ability to work through learning, usually by changing his attitude and developing his skills and understanding.

Cont..

Stage-5: Product Testing and Integration



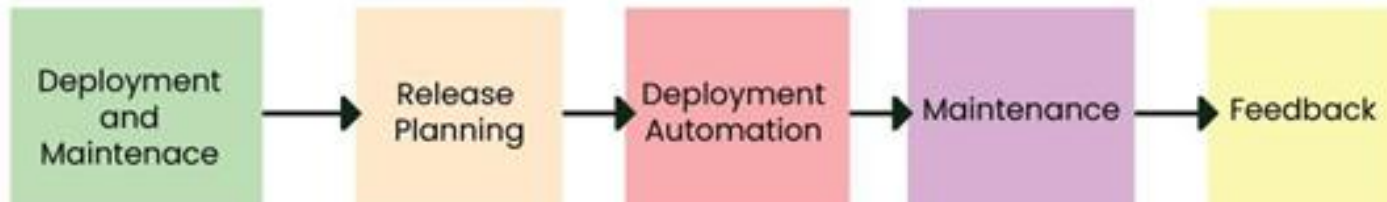
Cont..

Stage-6: Deployment and Maintenance of Products

- After detailed testing, the conclusive product is released in phases as per the organization's strategy.
- Then it is tested in a real industrial environment. It is important to ensure its smooth performance. If it performs well, the organization sends out the product as a whole.
- After retrieving beneficial feedback, the company releases it as it is or with auxiliary improvements to make it further helpful for the customers.
- However, this alone is not enough. Therefore, along with the deployment, the [product's supervision](#).

Cont..

Stage 6: Deployment and Maintenance of Products



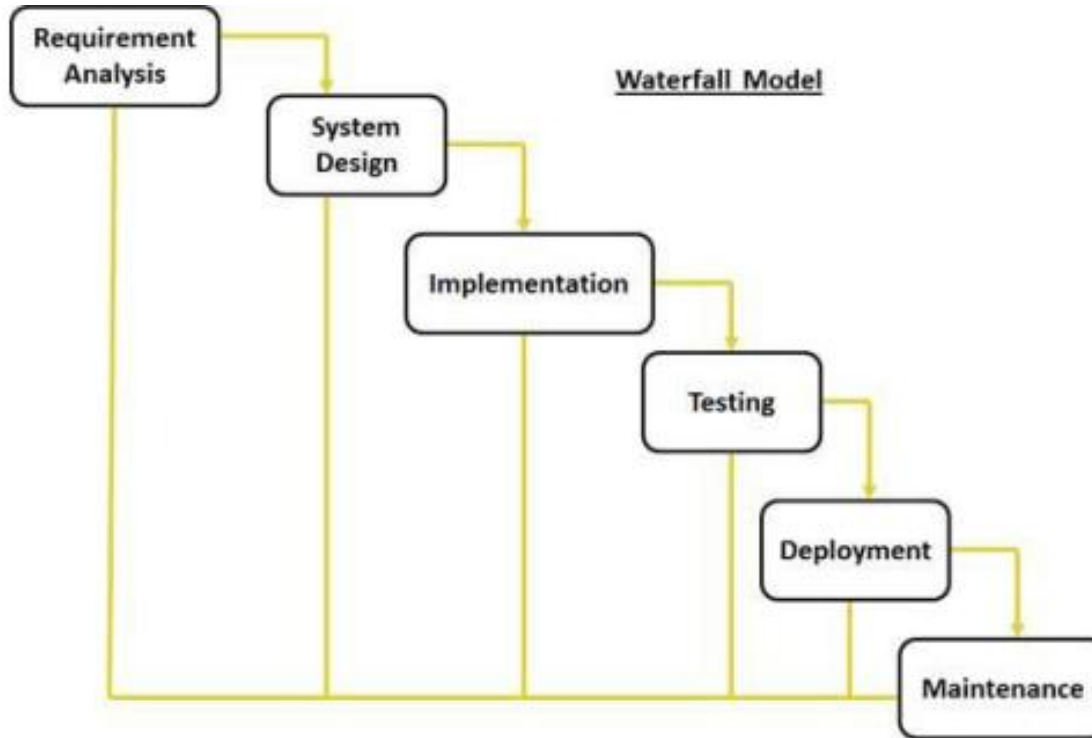
Software Development Life Cycle Models

- The most important and popular SDLC models are given below.

A. Waterfall Model

- It is the first software development process model.
- Waterfall model is a sequential process model which does not overlap. It means that until the one phase is not completed then next phase cannot start.
- It is simple and easy to understand.

Software Development Life Cycle Models



Cont..

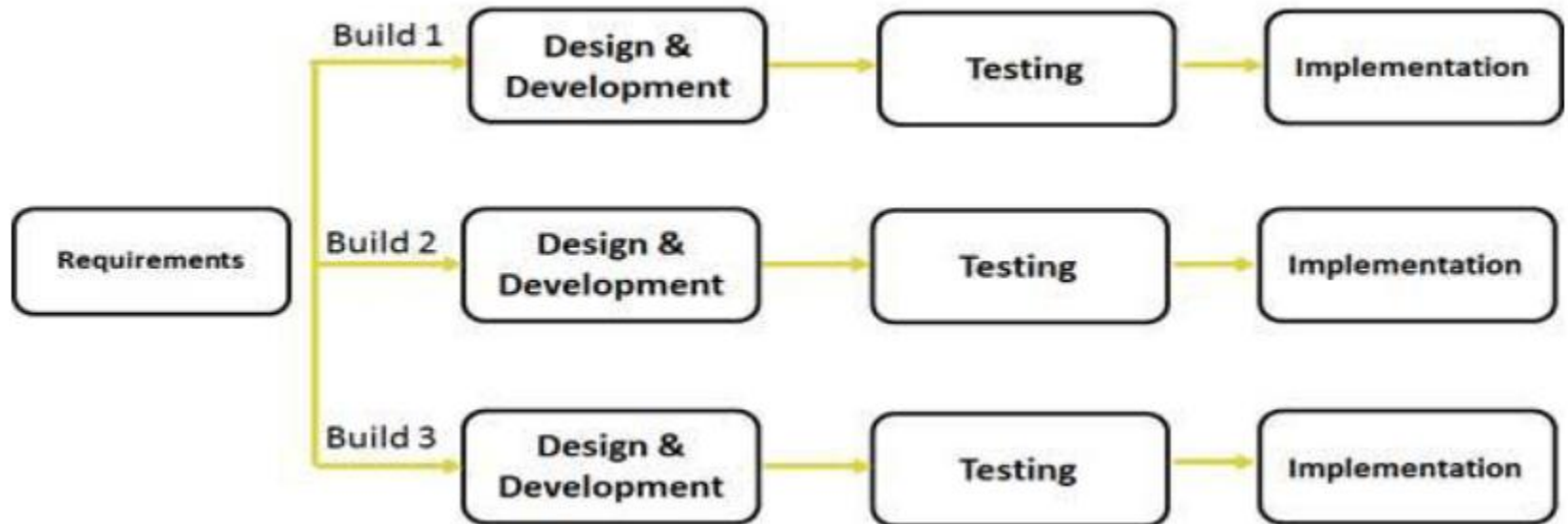
Advantages	Disadvantages
• It is very easy to understand and use.[4] • Each phase has a specific deliverable and review process.[5] • Phases are processed and completed one at a time. • Works well for small projects where requirements are well understood. • Clearly defined phases. • Well understood deliverables. • Arranging tasks easily. • Process and results are documented.	• Working software is available late during the life cycle. • It has a lot of risks. • A poor model for large projects. • It is not fit for projects where requirements are changed frequently.[4] • Difficult to measure progress during phases. • It cannot manage changing requirements. • Adjusting scope during the process model can end a project.

Cont..

B. ITERATIVE MODEL

- In iterative model, requirements are not completed and started iterative process with a small set of requirements.
- Each iteration evolves a small version of product and it is repeated until the final version is developed.
- Iterative process model starts implementation with a subset of requirement specifications.
- Each iteration is added new functionality in the process and continuous until it is completed.

Cont..



Cont..

SPIRAL PROCESS MODEL

- This SDLC model is the combination of iterative model and sequential model like waterfall model.
- The spiral model combines the idea of iterative and waterfall development in a very systematic and controlled way.

Cont..

- Spiral model consists of four models which are:

1) Identification

The purpose of this phase is to collect business requirements. The system requirements, subsystem requirement and unit requirements are all done in this initial phase. For this purpose continuous communication with customer is done and at the end system is deployed in the customer's environment.

2) Design

In this phase, architectural design, logical design of modules, physical design and final design of subsequent spirals are included.

3) Construction

In this phase, actual development is started. In baseline phase only proof of concept is developed to get customer feedback. When requirements and design details are cleared then a working model is developed which is called build and send this build to the customer for further feedback.

Cont..

4) Evaluation and Risks

- In this phase, identifying, estimating and monitoring technical feasibility and management risk for example schedule slippage and cost overrun.
- The build developed in first iteration is sent to customer for evaluation and feedback. The next development starts based on the customer feedback in the next iterations. The iteration process continuous through whole life cycle.

5) Spiral Model Application

- Spiral model is widely used in the industry. The situations in which spiral model are use are given below:
 - Constraints on budget and risk evaluation are important.
 - Medium to high projects.
 - Where long project commitment due to changing requirement environment.
 - When customer does not know his complete requirements.
 - Requirements are complexes and need evaluation to get clarity.
 - Where changes are expect in during life cycle.

Cont..

Agile Model

- The agile model in SDLC was mainly designed to adapt to changing requests quickly.
- The main goal of the Agile model is to facilitate quick project completion.
- The agile model refers to a group of development processes. These processes have some similar characteristics but also possess certain subtle differences among themselves.

Cont..

- **V-Shaped Model**
- The V-shaped model in **SDLC** is executed in a sequential manner in V-shape.
- Each stage or phase of this model is integrated with a testing phase. After every development phase, a testing phase is associated with it, and the next phase will start once the previous phase is completed, i.e., development & testing.
- It is also known as the verification or validation model.

Cont..

- **Big Bang Model**
- The [Big Bang model](#) in SDLC is a term used to describe an informal and unstructured approach to software development, where there is no specific planning, documentation, or well-defined phases.

Cont..

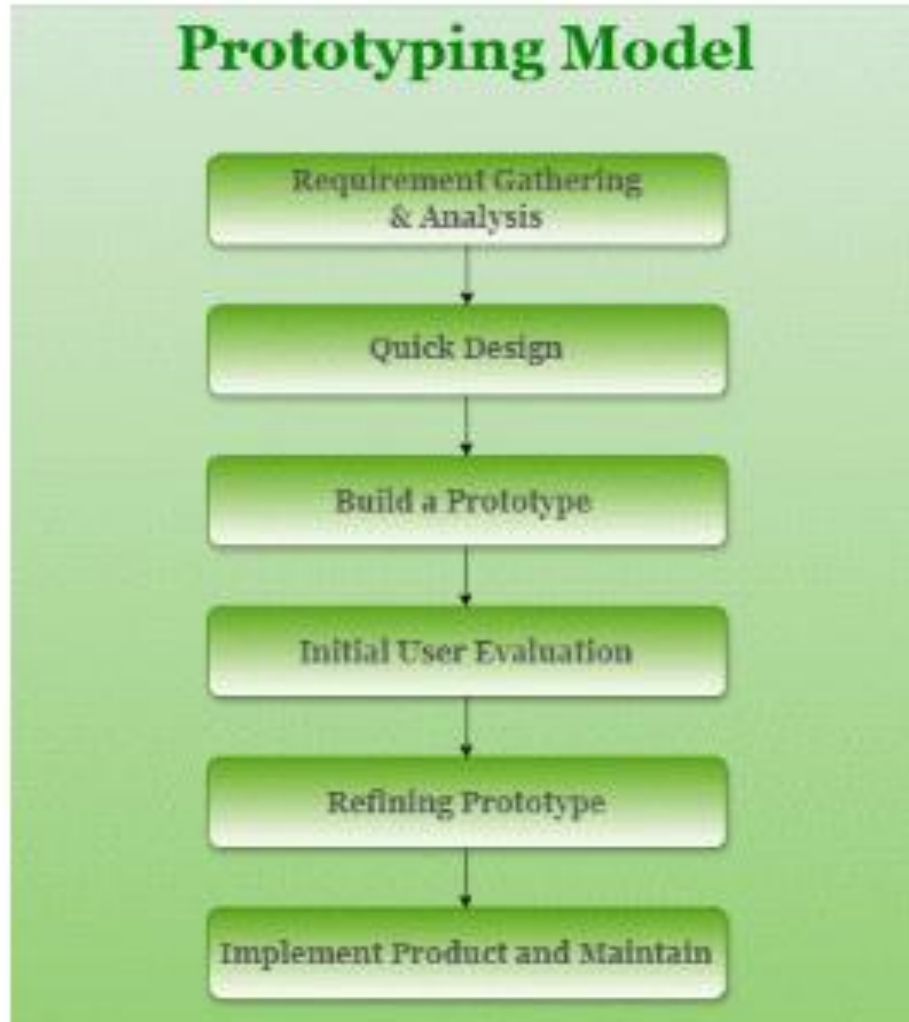
Prototyping Model

- The Prototyping Model is one of the most popularly used [Software Development Life Cycle Models \(SDLC models\)](#).
- This model is used when the customers do not know the exact project requirements beforehand.
- In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

Cont..

- In this process model, the system is partially implemented before or during the analysis phase thereby allowing the customers to see the product early in the life cycle.
- The process starts by interviewing the customers and developing the incomplete high-level paper model.
- This document is used to build the initial prototype supporting only the basic functionality as desired by the customer.
- Once the customer figures out the problems,
- the prototype is further refined to eliminate them.
- The process continues until the user approves the prototype and finds the working model to be satisfactory.

Cont..



How to Choose an SDLC Model?

- Choosing the right SDLC (Software Development Life Cycle) model is essential for project success. Here are the key factors to consider:
- **Project Requirements:**
 - **Clear Requirements:** Use [Waterfall](#) or **V-Model** if requirements are well-defined and unlikely to change.
 - **Changing Requirements:** Use **Agile** or **Iterative** models if requirements are unclear or likely to evolve.
- **Project Size and Complexity:**
 - **Small Projects:** Use **Waterfall** or **RAD** for small, simple projects.
 - **Large Projects:** Use **Agile**, **Spiral**, or **DevOps** for large, complex projects that need flexibility.

Cont..

- **Team Expertise:**
 - **Experienced Teams:** Use **Agile** or **Scrum** if the team is familiar with iterative development.
 - **Less Experienced Teams:** Use **Waterfall** or **V-Model** for teams needing structured guidance.
- **Client Involvement:**
 - **Frequent Client Feedback:** Use **Agile**, **Scrum**, or **RAD** if regular client interaction is needed.
 - **Minimal Client Involvement:** Use **Waterfall** or **V-Model** if client involvement is low after initial planning.

Cont..

- **Time and Budget Constraints:**
 - **Fixed Time and Budget:** Use **Waterfall** or **V-Model** if you have strict time and budget limits.
 - **Flexible Time and Budget:** Use **Agile** or **Spiral** if you can adjust time and budget as needed.
- **Risk Management:**
 - **High-Risk Projects:** Use **Spiral** for projects with significant risks and uncertainties.
 - **Low-Risk Projects:** Use **Waterfall** for projects with minimal risks.

Cont..

- **Product Release Timeline:**
 - **Quick Release Needed:** Use **Agile** or **RAD** to deliver products quickly.
 - **Longer Development Time:** Use **Waterfall** or **V-Model** for projects with no urgent deadlines.
- **Maintenance and Support:**
 - **Long-Term Maintenance:** Use **Agile** or **DevOps** for projects needing continuous updates and support.
 - **Minimal Maintenance:** Use **Waterfall** or **V-Model** if little future maintenance is expected.

Cont..

- **Stakeholder Expectations:**
 - **High Stakeholder**
Engagement: Use **Agile** or **Scrum** if stakeholders want ongoing involvement.
 - **Low Stakeholder**
Engagement: Use **Waterfall** or **V-Model** if stakeholders prefer involvement only at major milestones.

summarize

- **Waterfall:** *Best for clear, stable projects with minimal changes.*
- **V-Model:** *Good for projects with clear requirements and a strong focus on testing.*
- **Agile/Scrum:** *Ideal for projects with changing requirements and frequent client interaction.*
- **Spiral:** *Suitable for high-risk projects with evolving requirements.*
- **RAD:** *Useful for projects needing rapid development.*
- **DevOps:** *Best for continuous integration and ongoing support*

Process Metrics

- Process Metrics are the measures of the development process that create a body of software.
- A common example of a **process metric** is the length of time that the process of software creation tasks.
- Based on the assumption that the quality of the product is a direct function of the process, process metrics can be used to estimate, monitor, and improve the reliability and quality of software. ISO- 9000 certification, or “**Quality Management Standards**“, is the generic reference for a family of standards developed by the **International Standard Organization (ISO)**.

Types of Process Metrics



Cont..

- **Static Process Metrics:** Static Process Metrics are directly related to the defined process.
 - For example, the number of types of roles, types of artifacts, etc.
- **Dynamic Process Metrics:** Dynamic Process Metrics are simply related to the properties of process performance.
 - For example, how many activities are performed, how many artifacts are created, etc.)
- **Process Evolution Metrics:** Process Evolution Metrics are related to the process of making changes over a period of time.
 - For example, how many iterations are there within the process)

Top 7 Process Metrics

- **Lead Time:** Lead Time measures the time taken from initiating a process (such as starting work on a task) to its completion (finishing the task).
 - It indicates how quickly work moves through the development process.
- **Cycle Time:** Cycle Time tracks the duration it takes to complete one full cycle of a process, from beginning to end.
 - It provides insights into the efficiency and effectiveness of the development workflow.
- **Throughput:** Throughput basically Quantifies the rate at which tasks or features are completed within a given timeframe.
 - It reflects the productivity and capacity of the development team.
- **Work in Progress (WIP):** [Work in Progress \(WIP\)](#) indicates the number of tasks or features currently being worked on but not yet completed.
 - It helps in identifying bottlenecks and managing workflow to ensure tasks are completed efficiently.

Cont..

- **Defect Density:** Defect Density measures the number of defects or bugs found per unit of work or code.
 - It helps in assessing the quality and reliability of the software being developed.
- **Process Efficiency:** Process Efficiency evaluates the ratio of value-added work (tasks that directly contribute to delivering value to the customer) to non-value-added work (tasks that do not directly contribute to value delivery).
 - It identifies opportunities for streamlining processes and reducing waste.
- **Process Compliance:** Process Compliance assesses the extent to which development processes adhere to defined standards, guidelines, or regulations.
 - It ensures consistency and quality in the software development process.

Object oriented system development methodology

Why an object oriented?

- In OO, systems are easier to adapt to **changing requirements**, easier to **maintain**, more **robust**, and promote greater **design** and code reuse.
 - 1 **Modularity** : system analysts can break complex system into smaller, and more manageable modules
 - 2 **Higher level of abstraction** : The OO approach supports abstraction at the object level Since objects **encapsulate** both data and methods, they work at higher level of abstraction makes designing, coding, and maintaining the system much easier.

Promotion of reusability

- Via inheritance, **classes** can be **built** from each other Only differences and enhancements between the classes need to be designed and coded
- All previous functionality (and data) remains and can be used without change.
- The real advantage of using an OO-approach is that you can build on **what you already have**.

Overview of the unified approach

- **unified approach(UA)** is a methodology for software development that **combines** the best **practices, processes,** and guidelines along with the UML.
- UML is a set of **notations and conventions** used to describe and model an **application independent of the development** methodology or programming language
- It is a **specific methodology** that maps out **when** and **how** to use the various **Unified Modeling Language (UML) techniques** for object-oriented analysis and design
- **unified modeling language (UML)** is a set of notations and conventions used to describe and model an application.
- Applying past development efforts to future projects will improve the quality of the product and reduce the cost and development time

- The **Unified Process** is a **two-dimensional** systems development process described by a set of phases and workflows.
- It consists of five core workflows: **requirements, analysis, design, implementation, test** and four phases: **inception, elaboration, construction, and transition**.
- The **phases** describe how an **information system** evolves through time.
- **A workflow** is defined as a **sequence of activities** that produces a result of observable value.

- Each cycle goes through all **four phases** and addresses the development of a core workflow:
- **Requirement**: the product is the use case model
- **Analysis**: the product is the analysis model
- **Design**: the product is the design and the architecture model
- **Implementation**: the product is the implementation model
- **Test**: the product is the test model
- The life cycle phase of the UP are as follows
 - Inception: the **seed idea** is developed
 - Elaboration: the software **design** or **architecture** is defined
 - Construction: the software is **built**
 - Transition: the software is turned over to the user

Phases of Unified Process

- The phases in Unified Process provide a structured framework for managing the various activities and tasks involved in building a software system. Here's an in-depth look at each phase:

1. Inception

- This is the initial phase where the project's scope, objectives, and feasibility are determined.
- Key activities in this phase include identifying stakeholders, defining the initial requirements, outlining the project plan, and assessing risks.
- The goal of this phase is to establish a solid foundation for the project and ensure that it is worth pursuing.

2. Elaboration

- In this phase, the project requirements are analyzed in more detail, and the architecture of the system is defined.
- Key activities include developing use cases, creating the architectural baseline, identifying key components, and refining the project plan.
- The goal of this phase is to mitigate major risks and establish a solid architectural foundation for the project.

3. Construction

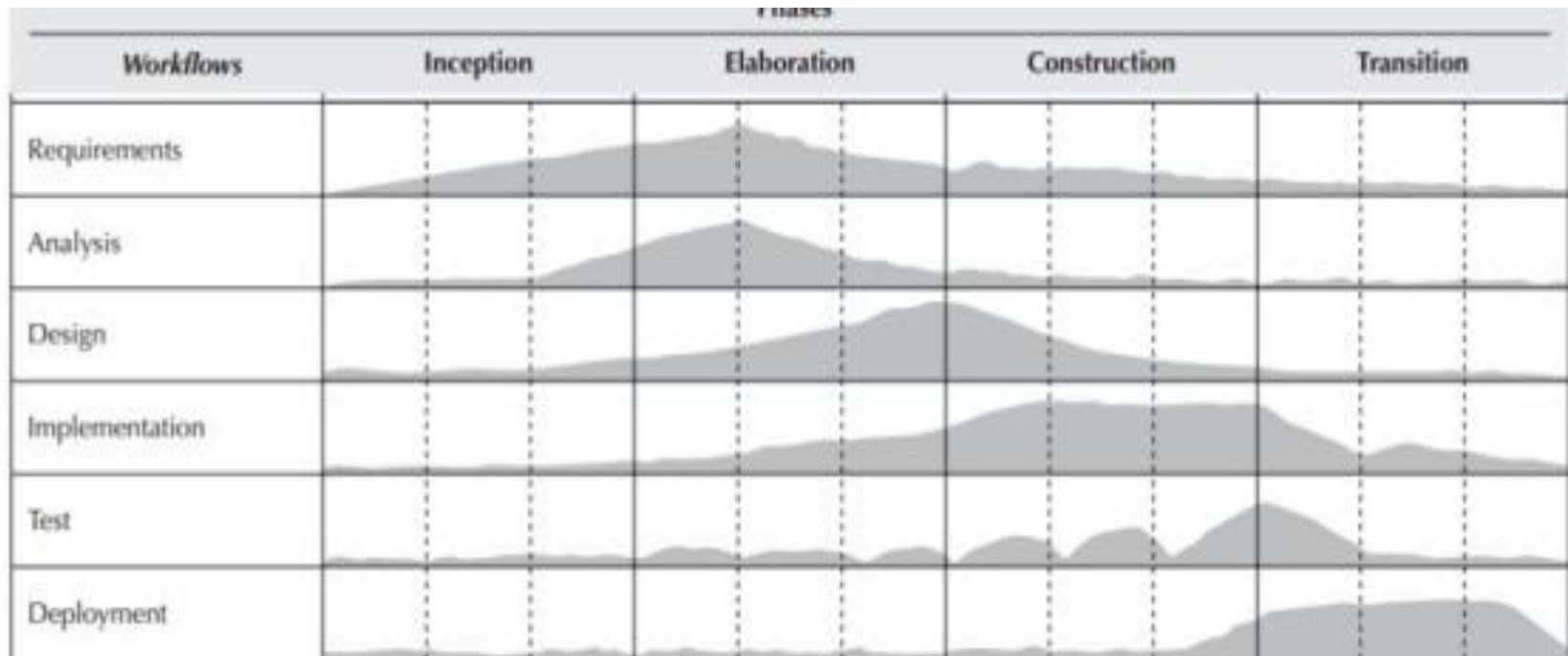
- This is the phase where the actual implementation of the system takes place.
- Key activities include developing, testing, and integrating the system components, as well as continuously verifying that the system meets the requirements.
- The goal of this phase is to build a complete, high-quality software product that is ready for deployment.

4. Transition

- In this final phase, the software is deployed to end users.
- Key activities include user training, final system testing, and transitioning the system to the operations and maintenance team.
- The goal of this phase is to ensure a smooth transition from development to production and to address any issues that arise during deployment.

- These phases are iterative, meaning that they may be revisited multiple times throughout the project to incorporate feedback, make improvements, and address changes in requirements.
- This iterative approach allows for flexibility and adaptability, making the Unified Process well-suited for complex and evolving software projects.

- The Unified Process is a two-dimensional systems development process described by a set of phases and workflows.



Object oriented philosophy

The Object-Oriented Philosophy

- Every program is really a simulation.
- The quality of a program's design is proportional to the faithfulness with which the structures and interactions in the program mirror those in the real world.
- According to this philosophy, a program to manage book handling in a library is really a simulation of what the librarians would have done prior to automation.
- A program to compute a company payroll is a simulation of what the accounting clerks would have done.
- A program to compute student grades at the end of a semester is a simulation of what a teacher would have done manually.
- Now, sometimes the worlds being simulated may be artificial or imaginary (e.g., chess and other games, fractal mathematics) but, as long as they are well understood, that really doesn't matter.

- The OO philosophy suggests that the things manipulated by the program should correspond to things in the real world.
 - They should carry the same names as those things carry in the real world.
 - And they should interact in ways like those objects in the real world.

Basic concepts of an Object, Attributes of an object, its state and properties.

- An object is a representation of a specific entity in the real world May be tangible (physical entity) or intangible
An object is characterized by
 - Identity , state, behavior
- **Identity:-** is the property of an object that distinguishes it from all others. We commonly denote identity by names, such as Steve, George,
 - in programming, x, y
- references that distinguish without names: this, that,
 - in programming, *p

❖ **Properties (or Attributes):** It describes the state (or data) of an object

Example : car(Color, manufacturer, cost, model and owner).

Example2: Every car on the road has certain unique identity such as a license plate number or Vehicle Identification Number (VIN).

Basic concepts of an Object, Attributes of an object, its state and properties.

- **State:-** The state of an object consists of the properties of the object, and the current values of those properties.
- For any given moment, the state of an object is formed from its characteristics.
 - **Example:** The state of 'car' can include properties like color, speed, fuel level and present gear.
- **Behavior:**
- This is a terminology that refers to the actions or methods that are performed by an object. They describe how the given object relates with others or its environment.
 - **Example:** Behaviors for cars can include accelerate, brake, changeGear, turnLeft.
- ❖ **Methods (or Procedures):** It defines its **behavior** (or operations)
Example:(Go, stop, turn left, turn right) etc.

Key Points:

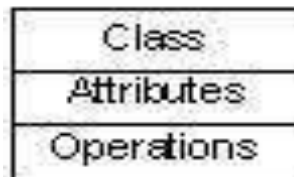
- **Encapsulation:** Objects encapsulate both state and behavior. The object encapsulates the internal state and uses methods to control access to it, promoting data security and integrity.
- **Abstraction:** Objects make real-world things seem less real by modeling their most important traits and interactions.
 - This lets developers focus on what is important and ignore the details that are not necessary.
- **Inheritance:** Through inheritance, objects can inherit attributes and behaviors from other objects, fostering code reuse and creating a hierarchical structure.
- **Polymorphism:** Objects of different classes can be treated as objects of a common base class through polymorphism, enabling flexibility and extensibility in the code.
 - Polymorphism means that the same operation may behave differently on different classes

Chapter 2: Unified Modeling Language (UML)

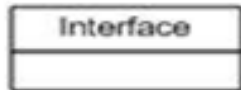
UML - Building Blocks

- As UML describes the real-time systems, it is very important to make a conceptual model and then proceed gradually.
- The conceptual model of UML can be mastered by learning the following three major elements –
 - UML building blocks
 - Rules to connect the building blocks
 - Common mechanisms of UML
- This title describes all the UML building blocks. The building blocks of UML can be defined as –
 - Things
 - Relationships
 - Diagrams

- **Things** are the most important building blocks of UML. Things can be –
 - Structural
 - Behavioral
 - Grouping
 - Annotational
- **Structural things:-** define the static part of the model. They represent the physical and conceptual elements.
 - **Class** – Class represents a set of objects having similar responsibilities.



- **Interface** – Interface defines a set of operations, which specify the responsibility of a class



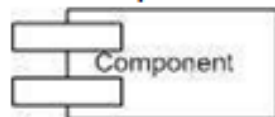
- **Collaboration** – Collaboration defines an interaction between



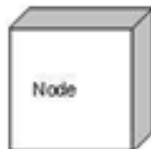
- **Use case** – Use case represents a set of actions performed by a system for a specific goal.



- **Component** – Component describes the physical part of a system.



- **Node** – A node can be defined as a physical element that exists at run time.



- **Actor** – We represent an actor in a UML diagram using a stick person notation.



Use Case Diagram

- A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals.
- It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.

Association

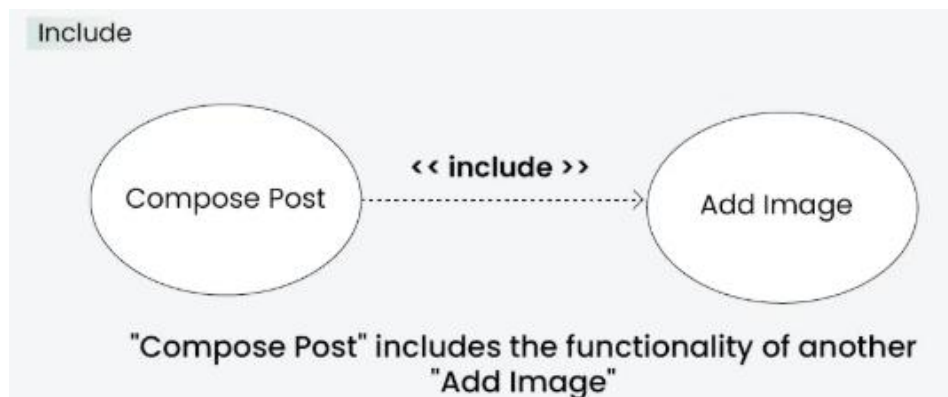


Represents a communication or interaction between an actor (Customer) and a use case (Transfer Funds)

2. Include Relationship

The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case.

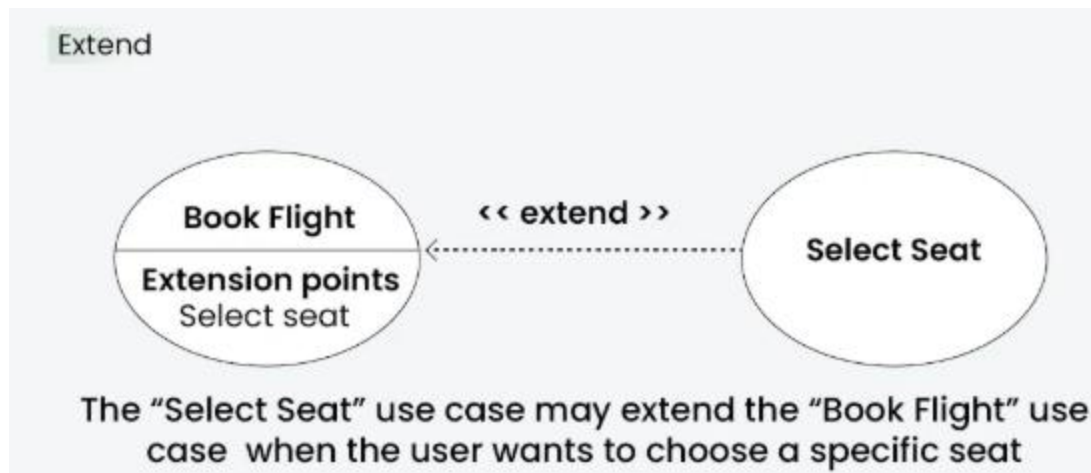
- **Example: Social Media Posting**
- **Use Cases:** Compose Post, Add Image
- **Include Relationship:** The "Compose Post" use case includes the functionality of "Add Image." Therefore, composing a post includes the action of adding an image.



- **Extend Relationship**

- The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the keyword "extend." This relationship is useful for handling optional or exceptional behavior.

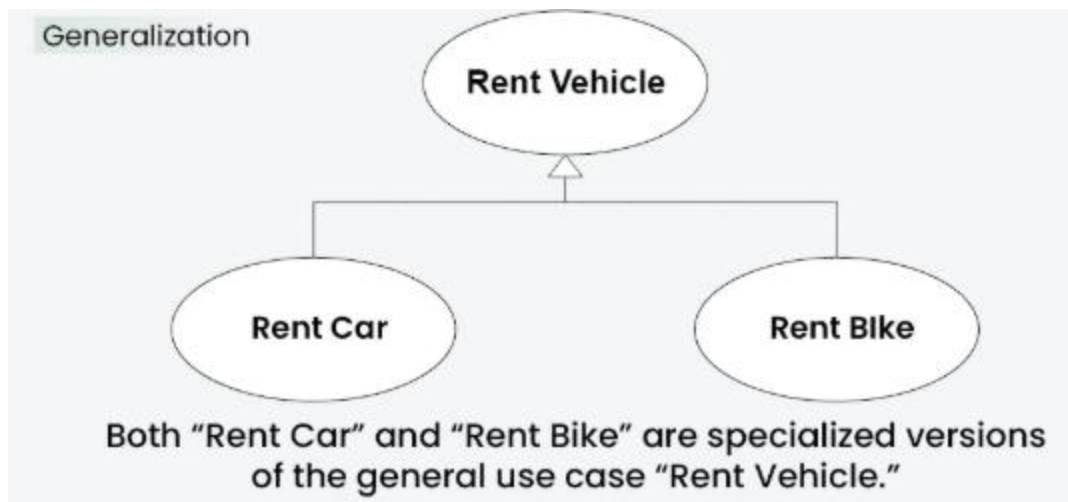
- **Eg: Flight Booking System**

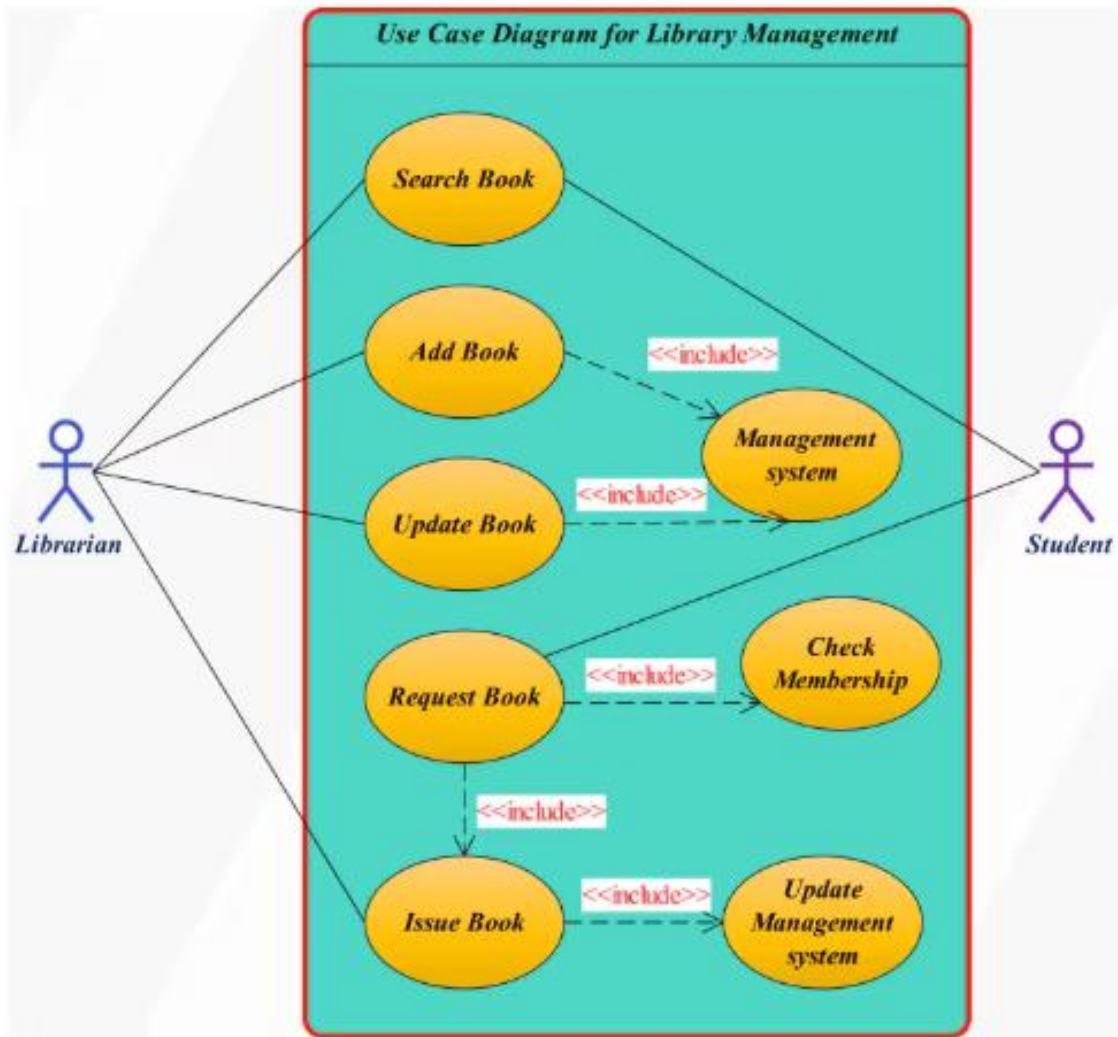


- **Generalization Relationship**
- The Generalization Relationship establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another.

Example: Vehicle Rental System

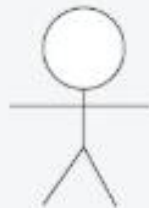
Generalization Relationship: Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."



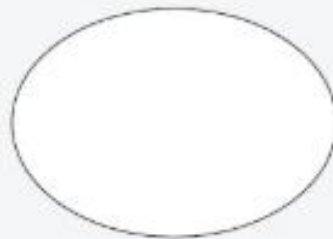


- **What are common Use Case Diagram Tools and Platforms?**
- Several tools and platforms are available to create and design Use Case Diagrams.
- **Lucidchart:**
 - Cloud-based collaborative platform.
 - Real-time collaboration and commenting.
 - Templates for various diagram types.
- **draw.io:**
 - Free, open-source diagramming tool.
 - Works offline and can be integrated with Google Drive, Dropbox, and others.
 - Offers a wide range of diagram types, including Use Case Diagrams.
- **Microsoft Visio:**
 - Part of the Microsoft Office suite.
 - Supports various diagram types, including Use Case Diagrams.
 - Extensive shape libraries and templates.
- **SmartDraw:**
 - User-friendly diagramming tool.
 - Templates for different types of diagrams, including Use Case Diagrams.
 - Integration with Microsoft Office and Google Workspace.
- **PlantUML:**
 - Open-source tool for creating UML diagrams.
 - Text-based syntax for diagram specification.
 - Supports collaborative work using version control systems.

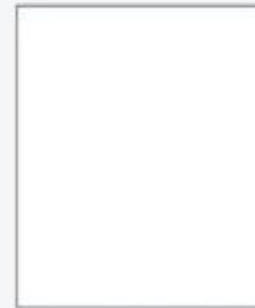
Use Case Diagram Notations



Actor



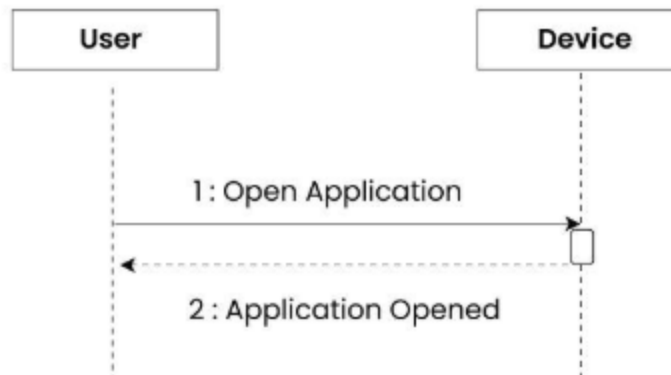
UseCase



System Boundary

- **What are Sequence Diagrams?**
- Sequence diagrams are a type of UML (Unified Modeling Language) diagram that visually represent the interactions between objects or components in a system over time. They focus on the order and timing of messages or events exchanged between different system elements.
- **Sequence Diagram Notations**
- **1. Actors**
- **2. Lifelines-** We display a lifeline in a rectangle called head with its name and type.
- **3. Messages-** The messages appear in a sequential order on the lifeline.
- We represent messages using arrows.
- Messages can be broadly classified into the following categories:
- **1. Synchronous messages**
- A synchronous message waits for a reply before the interaction can move forward.
- We use a **solid arrow head** to represent a synchronous message.

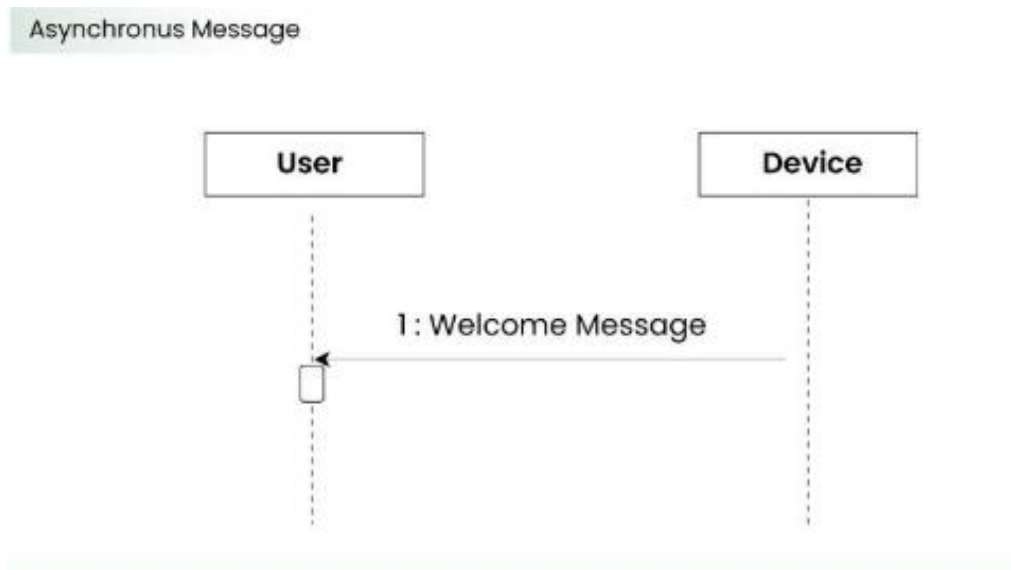
Synchronous Message



- **2. Asynchronous Messages**

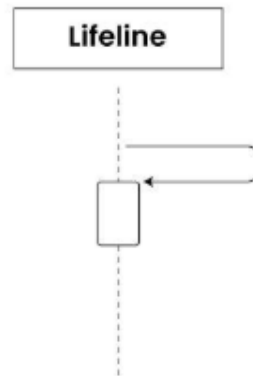
- An asynchronous message does not wait for a reply from the receiver.

We use a lined arrow head to represent an asynchronous message.



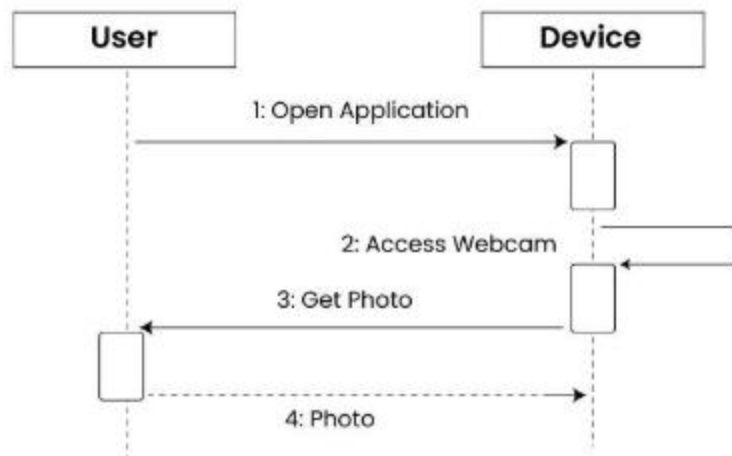
- **Self Message**
- Certain scenarios might arise where the object needs to send a message to itself. Such messages are called Self Messages and are represented with a **U shaped arrow**.

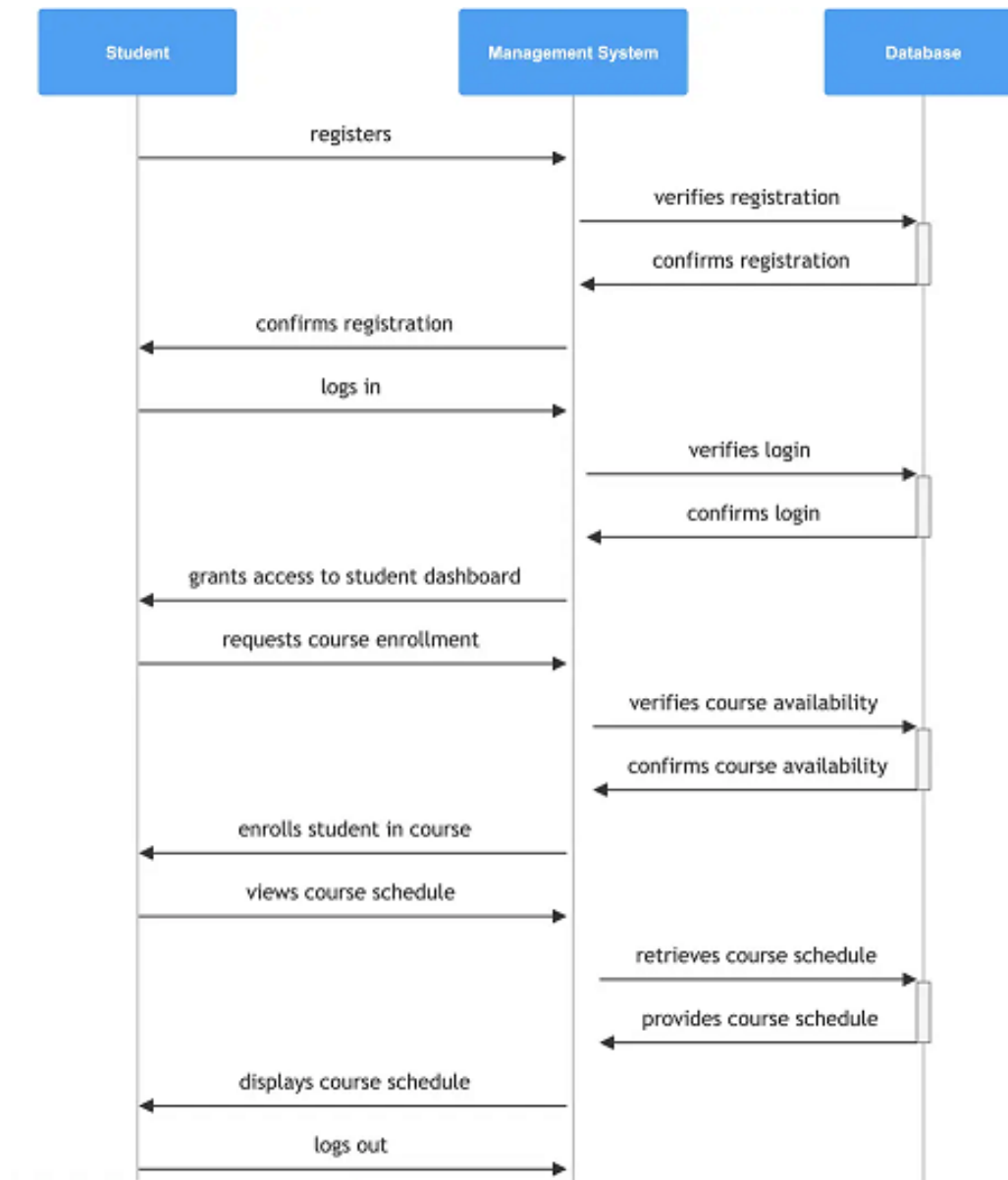
Self Image



- **Reply Message**
- Reply messages are used to show the message being sent from the receiver to the sender. We represent a return/reply message using an **open arrow head with a dotted line**. The interaction moves forward only when a reply message is sent by the receiver.

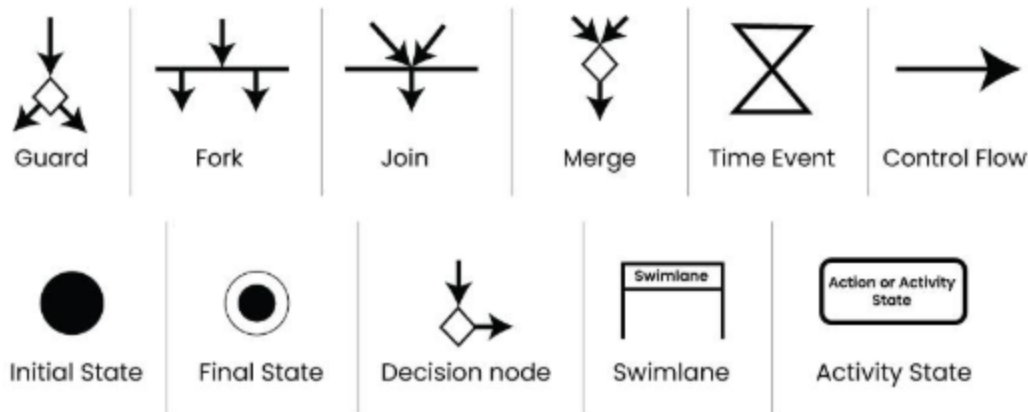
Reply Message Example



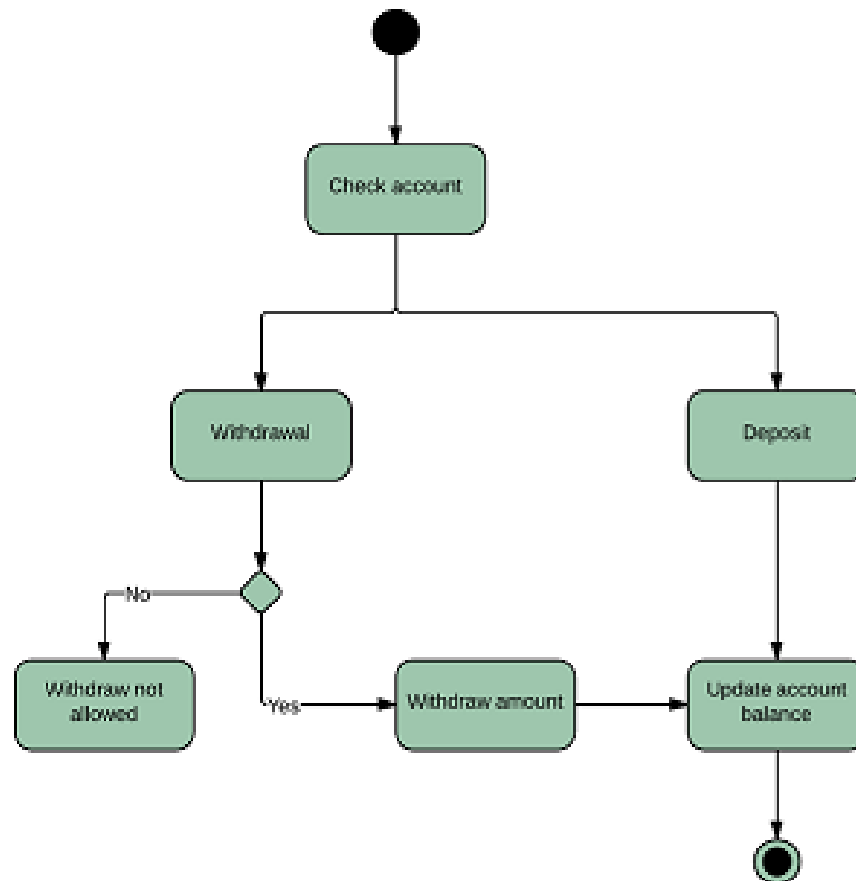


What is an Activity Diagram?

Activity diagrams show the steps involved in how a system works, helping us understand the flow of control.



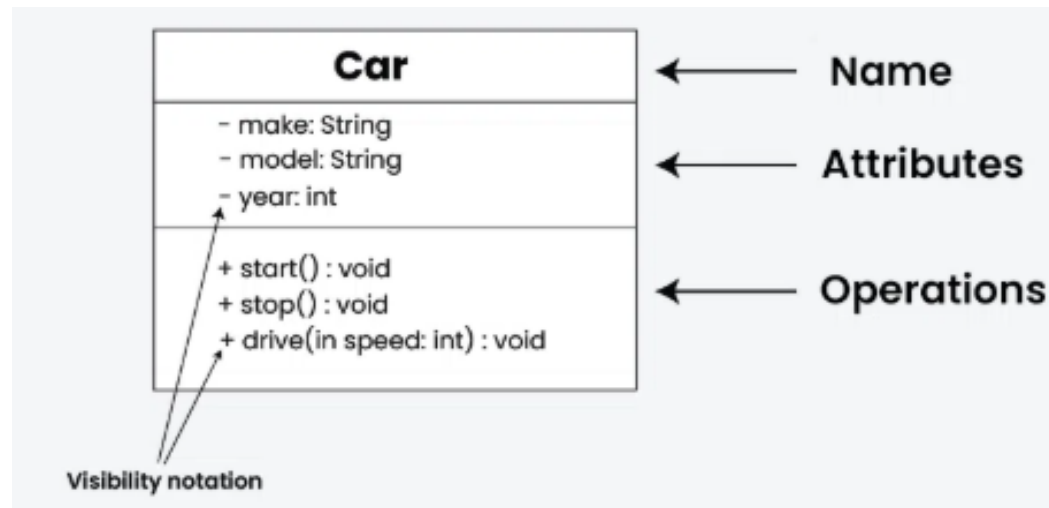
Activity diagram for a banking system



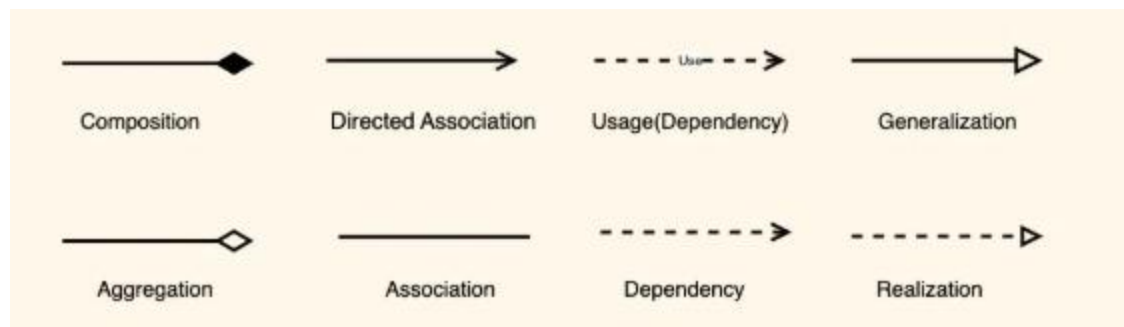
What are class Diagrams?

- Class diagrams are a type of UML (Unified Modeling Language) diagram used in software engineering to visually represent the structure and relationships of classes within a system i.e. used to construct and visualize object-oriented systems.
- Class diagrams provide a high-level overview of a system's design, helping to communicate and document the structure of the software.
- They are a fundamental tool in object-oriented design and play a crucial role in the software development lifecycle.
- **UML Class Notation**
- class notation is a graphical representation used to depict classes and their relationships in object-oriented modeling.

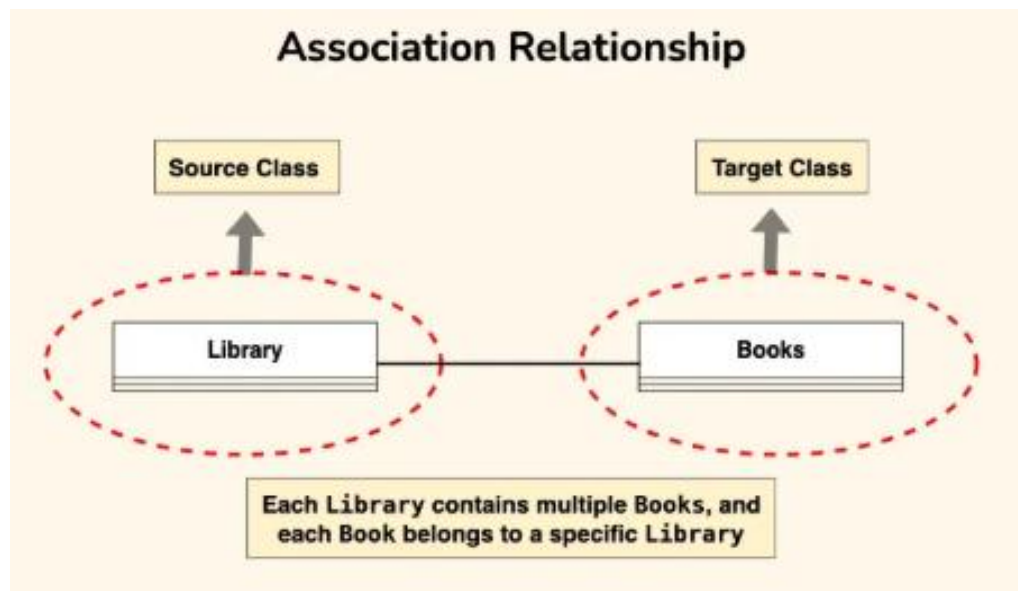
- **Class Name:**
 - The name of the class is typically written in the top compartment of the class box and is centered and bold.
- **Attributes:**
 - Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.
- **Methods:**
 - Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.
- **Visibility Notation:**
 - Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
 - + for public (visible to all classes)
 - - for private (visible only within the class)
 - # for protected (visible to subclasses)
 - ~ for package or default visibility (visible to classes in the same package)



- **Relationships between classes**
- In class diagrams, relationships between classes describe how classes are connected or interact with each other within a system.

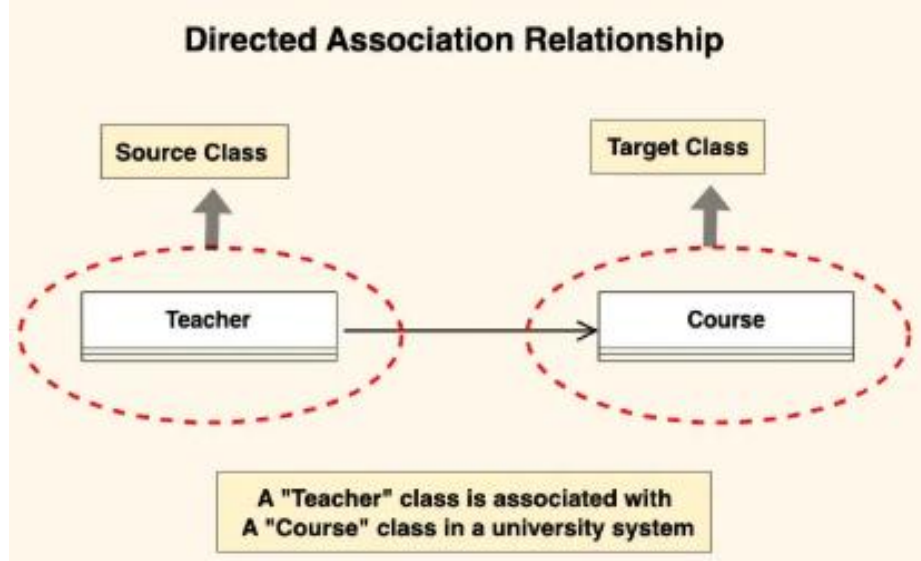


- **1. Association**
- An association represents a bi-directional relationship between two classes. It indicates that instances of one class are connected to instances of another class.



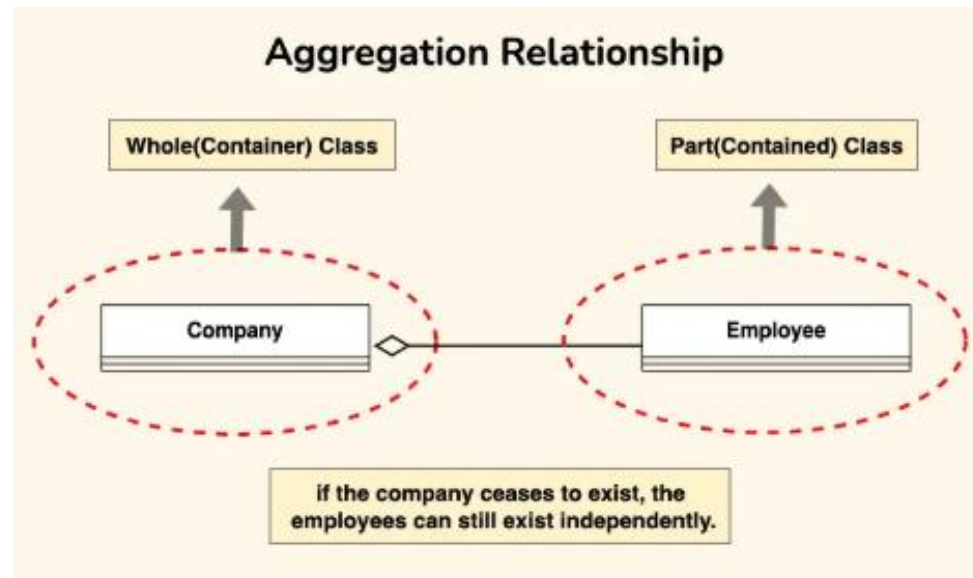
2. Directed Association

- where the association has a direction, indicating that one class is associated with another in a specific way.



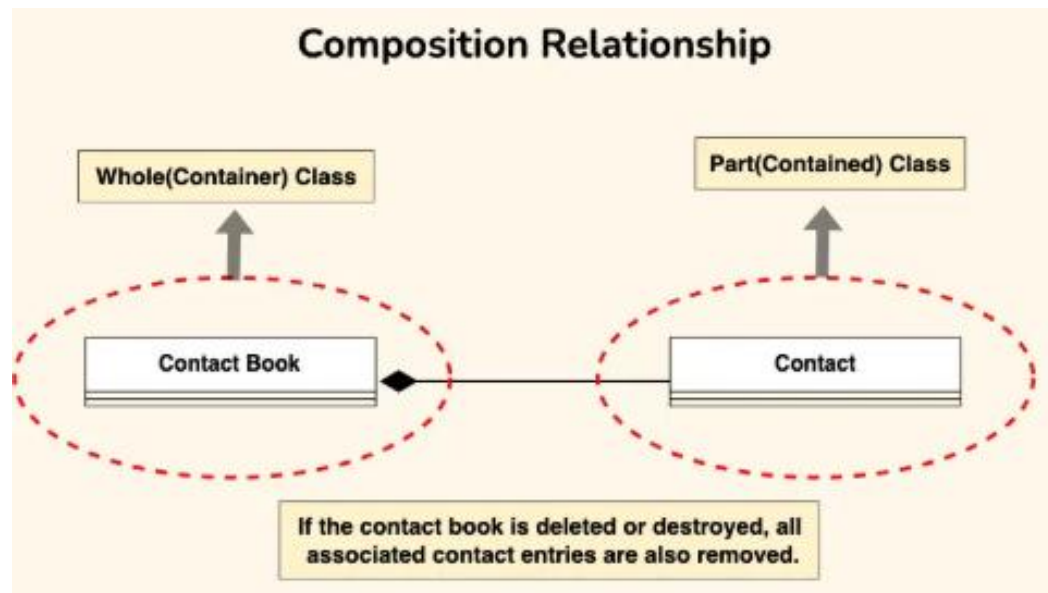
3. Aggregation

- Aggregation is a specialized form of association that represents a “whole-part” relationship.
- It denotes a stronger relationship where one class (the whole) contains or is composed of another class (the part).



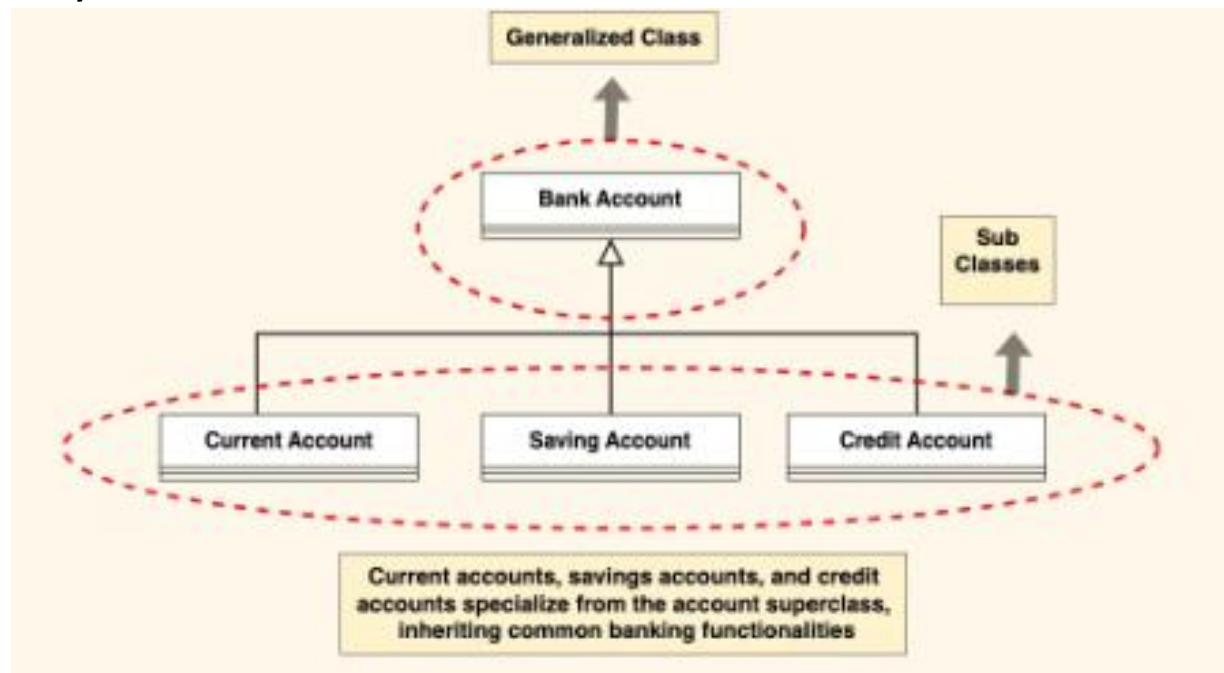
4. Composition

- Composition is a stronger form of aggregation, indicating a more significant ownership or dependency relationship.
- In composition, the part class cannot exist independently of the whole class.



5. Generalization(Inheritance)

- Inheritance represents an “is-a” relationship between classes, where one class (the subclass or child) inherits the properties and behaviors of another class (the superclass or parent).



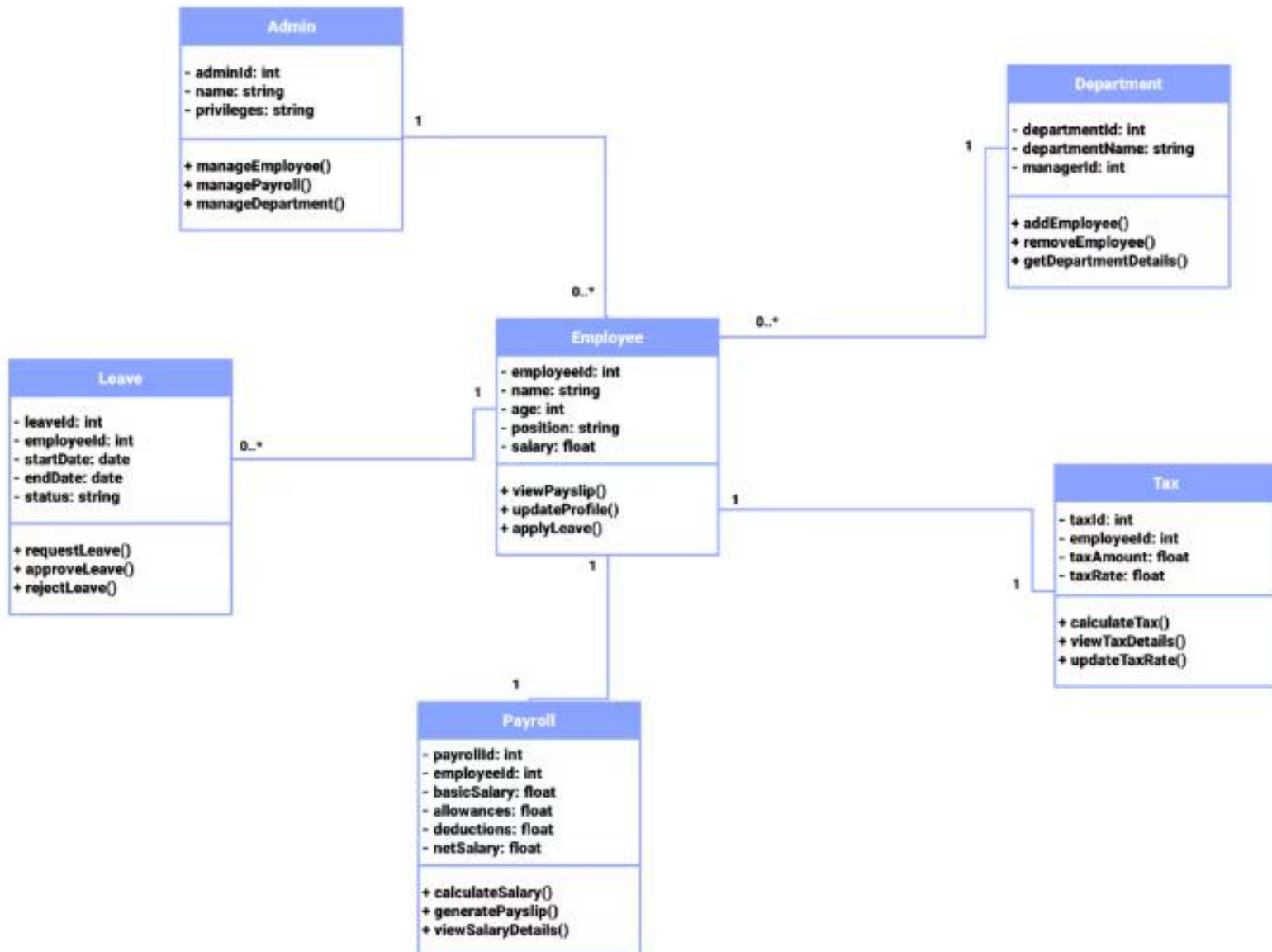
- A class has two aspects

1 Attribute:

- a named property of a class.
- values that instances of the class may hold

2 Operation:

- implementation of a service
- requested(any object of a given class)



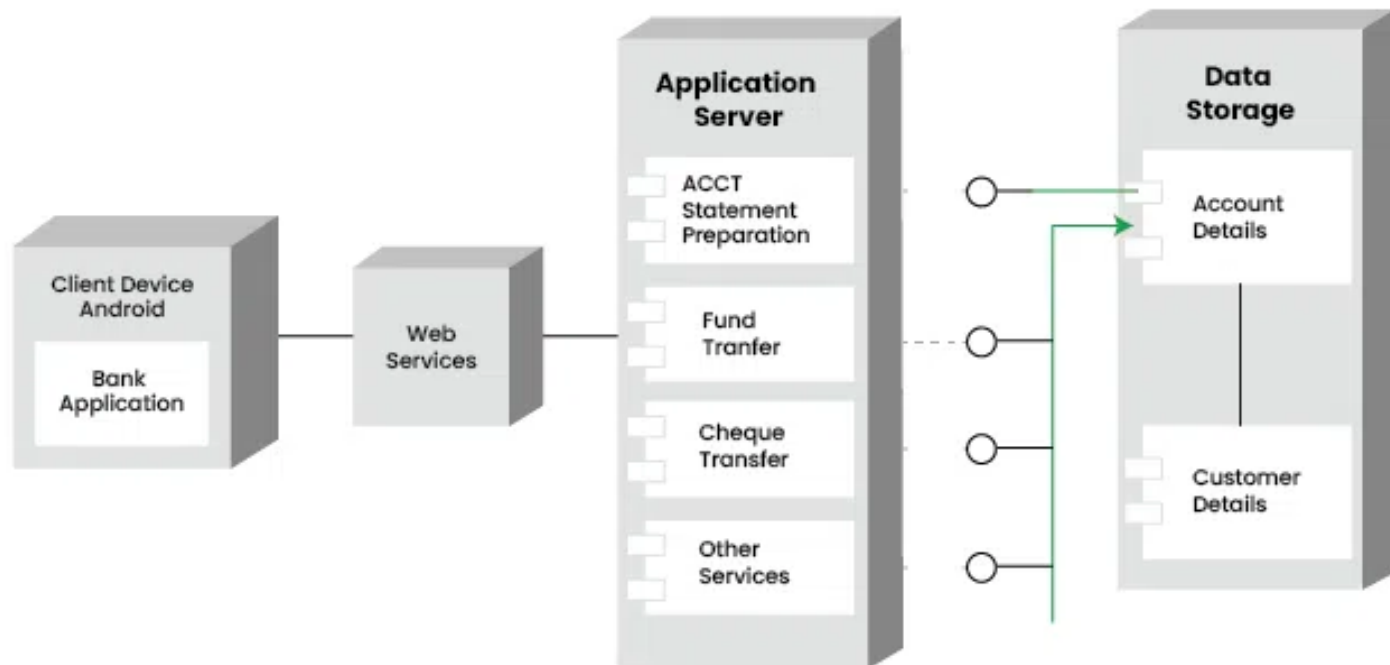
What is a Deployment Diagram?

- A Deployment Diagram shows how the software design turns into the actual physical system where the software will run.
- They show where software components are placed on hardware devices and shows how they connect with each other.
- This diagram helps visualize how the software will operate across different devices

Key elements of a Deployment Diagram

- **Nodes:** These represent the physical hardware entities where software components are deployed, such as servers, workstations, routers, etc.
- **Components:** Represent software modules or artifacts that are deployed onto nodes, including executable files, libraries, databases, and configuration files.
- **Artifacts:** Physical files that are placed on nodes represent the actual implementation of software components. These can include executable files, scripts, databases, and more.
- **Dependencies:** These show the relationships or connections between nodes and components
- **Associations:** Show relationships between nodes and components, signifying that a component is deployed on a particular node, thus mapping software components to physical nodes.
- **Deployment Specification:** including hardware specifications, software settings, and communication protocols.
- **Communication Paths:** Represent channels or connections facilitating communication between nodes and components and includes network connections, communication protocols, etc.

Deployment Diagram for Mobile Banking Android Services



Chapter 3

Requirements Elicitation

Definition of Requirements Elicitation

- Requirements Elicitation is a foundational process in software engineering where stakeholders' needs, desires, and constraints for a new or altered system are identified, gathered, and understood.
- It involves direct interaction with stakeholders, including users, customers, and others involved in the project, to capture detailed [system requirements](#).
- This process is not merely about asking what the stakeholders want but involves a deep investigation to uncover and document explicit, implicit, tacit, and future needs.

Stakeholders in Requirements Elicitation

- Stakeholders are individuals or groups with an interest in the outcome of the project.
- They can have an impact on or have an impact on the project's success. In requirement elicitation, stakeholders typically include:
- **End-users:** Who will directly interact with the system?
- **Business Managers:** Those who make strategic decisions based on the system's output.
- **Project Managers:** Who oversees the project's execution.
- **Development Team:** This includes software engineers, designers, and testers who build and maintain the system.
- **Customers:** Who commissions the software and funds the project?
- **Regulatory Bodies:** Whose standards and regulations must be met by the system.

Functional Requirements

What are Functional Requirements?

- These are the requirements that the end user specifically demands as basic facilities that the system should offer.
- These are represented or stated in the form of input to be given to the system, the operation performed and the output expected.
- They are the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

Eg:- Online Banking System

1. Functional Requirements:

- Users should be able to log in with their username and password.
- Users should be able to check their account balance.
- Users should receive notifications after making a transaction.

Functional Requirements

- **Authentication.** This group is about verifying the identity of a user before allowing access to the system, including entering usernames and passwords, biometric verification, or multifactor authentication.
- **Authorization levels.** These requirements aim to define and control the access levels of different users within a system. For example, an admin may have complete system access, while a regular user has limited access to certain features.
- **Data processing.** These requirements can include data entry, validation, storage, and retrieval.
- **User interface and user experience (UI/UX).** These are the requirements related to the design and interaction elements of the system. Their goal is to ensure that it's user-friendly and meets users' needs.
- **Reporting.** These requirements define generating reports, e.g., data sources, formats, etc.
- **System integration.** These requirements describe how the system interacts and integrates with other systems or third-party services.

Non-Functional Requirements

What are Non-Functional Requirements?

- These are the quality constraints that the system must satisfy according to the project contract.
- The priority or extent to which these factors are implemented varies from one project to another. They are also called non-behavioral requirements. They deal with issues like:
 - Portability
 - Security: ensure there is protection from unauthorized access to the system and its stored data.
 - [Maintainability](#)
 - [Reliability](#): defines how likely it is for the software to work without failure for a given time.
 - [Scalability](#): describe how the system must grow without negatively impacting its performance. Scalability has both hardware and software implications. For instance, you can increase scalability by adding memory, servers, or disk space.
 - Performance: describes the system's responsiveness to various user interactions.
 - Reusability
 - Flexibility

Eg:- Eg:- Online Banking System

Non-functional Requirements:

- The system should respond to user actions in less than 2 seconds.
- All transactions must be encrypted and comply with industry security standards.
- The system should be able to handle 100 million users with minimal downtime.

Requirement Validation

- The last phase of requirements engineering is requirements validation.
- Requirements validation is done to make sure that requirements are complete and consistent according to user requirements.
- The requirements validation process detects errors in the software requirements specification (SRS).
- Ambiguities and conflicts in requirements are resolved during requirements validation.

These phase includes few steps that checks the requirements, the steps are:

- **Consistency checks:-** The requirements should be consistent with other requirements, which means no two requirements should conflict or oppose each other.
- **Completeness checks:-** The requirements should be practically achievable.
 - The requirements should be complete in every sense.
- **Validity checks:- :-** The requirements must contain all relevant information
- **Realism checks:-** The requirements must address the actual needs of the system
- **Ambiguity checks:-** The requirements must be understandable by the stakeholders
- **Verifiability:-** Every requirement must be described in a way that precludes different interpretations.
- **Traceability:** the ability to describe and follow the life of a requirement, in both a forwards and backwards direction (i.e., from its origins, through its development and specification, to its subsequent deployment and use,